Pharmacy Ordering System (InfoPharma) Software

IPOS-PU (Online Platform)

Implementation Report

Document Title	MediFlow Solutions Implementation Report
Version	1.0
Team	Team 30
Subsystem	IPOS-PU (Online Platform)
Date	April 2026

Mediflow Solutions

MEDIFLOW

Table of Contents

..... 1

1 Introduction..... 3

1.1 Purpose and scope of the report..... 3

1.2 Implemented Packages..... 3

2 Software Compilation and Project Structure..... 3

2.1 Software Compilation..... 3

2.2 Project Structure..... 5

3 Run-Time Components and Deployment..... 6

3.1 Run-Time Components..... 6

3.2 Deployment..... 6

4 Testing Plan and Results..... 7

4.1 Unit Testing..... 7

4.2 Subsystem Testing..... 8

4.3 Required Interface Testing..... 9

4.4 Non-Functional Testing..... 10

Appendices..... 10

Appendix A..... 10

1 Introduction

1.1 Purpose and scope of the report

This report is the Implementation Report for the IPOS-PU (Online Platform) subsystem developed as part of the IN2033 Team Project. It covers how the system was built including how it is compiled, structured, deployed, and tested as well as the final design class diagram.

The IPOS-PU subsystem is an online platform built using Java, with JavaFX used for the user interface and SQLite for data storage. It allows members of the public to create accounts as either commercial or non-commercial users, browse products, and place orders. Commercial users must go through an approval process before they can fully access the system.

The subsystem also connects with other parts of the InfoPharma system including IPOS-SA for handling orders and account related processes and IPOS-CA for updating stock and fulfilling orders.

1.2 Implemented Packages

The following IPOS-PU packages have been implemented:

- IPOS – PU –MEMBERS: Handles user registration for commercial and non-commercial customers including account creation and login.
- IPOS – PU – SALES: Allows users to browse products, add items to a cart, and complete purchases. Orders are forwarded to IPOS-CA to update stock.
- IPOS – PU – COMMS: Manages communication with external services such as email notifications and payment processing.
- IPOS – PU – PRM: Supports creation and management of promotional campaigns including tracking user interactions.
- IPOS – PU – RPT: Generates reports such as sales data, promotion performance, and campaign activity.

2 Software Compilation and Project Structure

2.1 Software Compilation

Prerequisites

This section describes the technologies and tools required to compile and run the IPOS-PU subsystem.

Requirement	Details
IntelliJ IDEA	Used as the development environment to write, compile, and run the project.
Java Development Kit	Used to compile and run the Java application using the JVM.
Gradle	Manages dependencies and automates the build and compilation process.
JavaFX	Provides the graphical user interface components for the application.
SQLite	Stores application data in a lightweight, file-based database.
Javalin	Handles backend server logic and API requests.
JDBC (SQLite Driver)	Enables communication between the Java application and the database.

Dependency Management:

The project uses Gradle to automatically manage and install all required libraries, meaning users do not need to manually download any additional components. When the project is opened in IntelliJ IDEA, the Gradle wrapper automatically downloads the correct version of Gradle and configures all dependencies. IntelliJ will prompt a Gradle sync which must complete successfully before the application can be run.

The system relies on several external libraries to support its functionality, including:

- Javalin – for backend server functionality
- SQLite JDBC – for database connectivity
- Jackson Databind – for JSON processing

- SLF4J / Logback – for logging
- PDFBox – for report generation
- JavaFX libraries – for GUI components

This approach ensures that the project can be set up consistently across different machines without requiring manual configuration.

JavaFX Configuration

JavaFX is managed through Gradle using the OpenJFX plugin. The required libraries (version 21) are automatically downloaded and configured when the project is opened and synced in IntelliJ IDEA. The system uses the javafx.controls and javafx.fxml modules to provide the graphical user interface components.

Compilation and Execution Instructions

To compile and run the system, the following steps should be followed:

1. Open the project in IntelliJ IDEA
2. Allow Gradle to complete the initial dependency setup during project synchronisation
3. Ensure the project builds successfully with no errors
4. Navigate to the Main.java class located in the main package
5. Click the Run button in IntelliJ to start the application

When the system is executed:

- The Javalin backend server is started on port 8090
- The JavaFX graphical user interface is launched

Ensure that port 8090 is available before running the system.

2.2 Project Structure

The system is organised into packages based on functionality. Each package contains related classes responsible for a specific part of the subsystem.

Package	Classes	Responsibility
members	UserDatabase, UserType	Handles user account data and user type definitions
sales	Cart, CartItem, Merchant, Product, ProductFeed	Manages products, merchants, and shopping cart functionality

comms	CAClient, SAClient, Mail, MutateStockRequest, Payment, Response, StockItem, MailService, OrderService, PaymentService, RestServer	Handles communication with external subsystems, email, payments, and backend API services
prm	Promotion, PromotionDatabase	Manages promotions and campaign data
rpt	PdfExporter, ReportRow, ReportService	Generates reports and exports them as PDF files
ord	Order, OrderDatabase, OrderItem, OrderStatus	Handles order storage, order items, and order status management
data	Database, DatabaseUtility, PaymentDatabase	Manages database connections and shared data access operations
gui	AppView, SceneHelper, multiple controllers (e.g. LoginController, PromotionsController, HomeController, ReportsController)	Handles user interface logic and screen navigation

Additional supporting packages include **config** which stores application constants and **main** which handles application startup.

FXML Views

FXML files define the layout of the user interface screens, while controllers handle the logic behind them. The main FXML files used in the system include:

- login.fxml – user login screen
- register.fxml – user registration screen
- change-password.fxml – password update screen
- home.fxml – main dashboard screen
- checkout.fxml – checkout and payment screen
- orders.fxml – order history screen
- promotions.fxml – promotion management screen
- promotion-edit-popup.fxml – promotion editing popup
- reports.fxml – report display screen

The system uses JavaFX controls such as buttons, text fields, and table views to handle user input and display data. Data is stored in SQLite database tables including users, orders, promotions, and payment related data.

3 Run-Time Components and Deployment

3.1 Run-Time Components

The IPOS-PU subsystem consists of several run-time components that must be active for the system to function correctly. These components work together to provide the full online pharmacy ordering platform, including user interaction, business logic processing, data storage, and external system communication.

The main run-time components include:

JavaFX Frontend (GUI Layer)

The graphical user interface is built using JavaFX and is responsible for all user interactions. This includes authentication screens (login, registration, password management), the commercial user dashboard, product browsing, cart management, checkout, order tracking, promotions, and reporting interfaces. The GUI is structured using controllers located in the gui package, which handle user input and update views accordingly.

Javalin REST Server (Backend Layer)

The backend server is implemented using Javalin and is defined primarily in RestServer.java. This server runs on port 8090 and handles all HTTP requests between the frontend and backend services. It processes actions such as user authentication, order submission, payment processing, and data retrieval.

Database Layer (SQLite)

The system uses a local SQLite database managed through the data package, including classes such as Database.java, OrderDatabase.java, UserDatabase.java, and PromotionDatabase.java. This layer is responsible for persistent storage of users, products, orders, payments, and promotional data. The database is accessed through utility classes such as DatabaseUtility.java.

Service Layer (Business Logic)

Core business logic is handled in service classes such as:

OrderService

PaymentService

MailService

These services coordinate operations between the database, backend server, and external systems.

Inter-System Communication Clients

The system communicates with external subsystems using client classes located in the comms.client package:

SAClient – communicates with IPOS-SA (sales/account subsystem)

CAClient – communicates with IPOS-CA (courier/fulfilment subsystem)

Communication is typically handled via REST API calls.

External Libraries

The runtime also depends on several external libraries managed via Gradle:

Javalin (server framework)

SQLite JDBC (database connection)

Jackson (JSON processing)

PDFBox (report generation)

SLF4J / Logback (logging)

JavaFX modules (UI components)

Application Entry Point

The system is launched through Main.java, which initializes both:

The Javalin backend server

The JavaFX frontend application

3.2 Deployment

IPOS-PU is deployed as a JavaFX desktop client. The application is supported on all devices capable of hosting Java Runtime Environment 21 or later, namely Windows 10+, macOS, and Linux. It is distributed as a Gradle-built application bundle which the user installs locally. Because every instance runs on a member of the public's personal computer, the host machine sits outside InfoPharma's administrative control.

IPOS-PU uses SQLite, a file-based database, which removes the need for a separate database server. The database files sit in a ./data/ folder relative to the application's working directory and consist of two files: users.db, which holds user account data, and promotions.dB, which holds promotional campaign data. Both files are created automatically on first run if they do not already exist. Because the database is local to the machine running the client, each user holds their own independent copy of the data — a limitation of the v1 release which is addressed in the planned future evolution to a centralized, cloud-hosted database.

IPOS-PU integrates with two sibling subsystems being IPOS-SA and IPOS-CA using the Java interfaces defined in the design. The key interfaces involved are:

- I_IPOS-PU: The interface that the IPOS PU uses so that IPOS-SA can ask IPOS PU to send emails and process payments and generate reports.
- I_IPOS-CA: allows IPOS PU to update stock when someone makes an order,
- I_IPOS-SA_ProductCatalogue: lets IPOS PU read the product catalogue data.
- I_IPOS-SA_OrderManagement: lets IPOS PU submit and track orders.

External services are then handled by the COMMS package which controls the SMTP for emails and/or payment-processor connections. At deployment time all that is mentioned requires a stable network connection from the customers machine to the host where the subsystems and services are running. In the current v1 build, the inter-subsystem calls are in-process skelatons that are used for testing purposes mostly. A deployment will require the subsystems to be able to reach over the network at endpoints.

Category	Minimum	Recommended
Processor	Dual-core 1.6 GHz	Quad-core 2.0 GHz or higher
Memory (RAM)	2 GB	4 GB or higher
Free disk space	1 GB	5 GB
Operating system	Windows 10, macOS 11, or a modern 64-bit Linux	Windows 11, latest macOS, or later
Java	JRE 21	JRE 21 or later
Display	1280 × 720	1920 × 1080
Network	Stable broadband connection	Stable broadband connection

4 Testing Plan and Results

4.1 Unit Testing

Two key classes were selected for unit testing: Cart and Promotion. These were chosen because they contain the most important logic in the system which is managing what a user is buying and applying any active discounts to it. Tests were written using JUnit and run through Gradle. All 37 tests across 6 test classes passed successfully, as shown in the screenshot below and a report was generated.

Cart:

Test Case	Input	Expected Output	Actual Result
Add one item to cart	1x Ibuprofen (£4.00)	Cart size = 1	Pass
Add same item twice	2x Ibuprofen (£4.00)	Quantity = 2	Pass
Remove one of two items	2x then decrement	Quantity = 1	Pass
Remove only item	1x then decrement	Cart is empty	Pass
Clear cart after adding	1x Ibuprofen then clear	Cart is empty	Pass
Total price for two items	2x Ibuprofen (£4.00 each)	Total = £8.00	Pass
Total price on empty cart	No items	Total = £0.00	Pass

Promotion:

Test Case	Input	Expected Output	Actual Result
Promo within date range	Start 01/04/26, end 30/04/26, active=true	isCurrentlyActive = true	Pass
Promo already expired	Start 01/03/26, end 31/03/26	isCurrentlyActive = false	Pass
Promo disabled by admin	active=false	isCurrentlyActive = false	Pass
Promo starts next month	Start 01/05/26	isCurrentlyActive = false	Pass
10% off applied to	£10.00 with 10% discount	Purchase price =	Pass

Omega 3		£9.00	
50% off applied to Biotin	£20.00 with 50% discount	Purchase price = £10.00	Pass
Product not in promo	Ibuprofen, no discount entry	Full price = £5.00	Pass
No promo attached	Vitamin D, promo = null	Full price = £8.00	Pass
Promo fields stored correctly	April Deals, start/end April 2026	Getters return correct values	Pass

All Results

Gradle Test Run :test

Gradle Test Run :test

summary

37	0	0	1.443s	100%
tests	failures	skipped	duration	successful

Child	Name	Tests	Failures	Skipped	Duration	Success rate
CartTest	ac.csg.pu.CartTest	7	0	0	0.061s	100%
OrderItemTest	ac.csg.pu.ord.OrderItemTest	4	0	0	0.007s	100%
OrderTest	ac.csg.pu.ord.OrderTest	6	0	0	0.008s	100%
ProductTest	ac.csg.pu.ProductTest	4	0	0	0.006s	100%
PromotionTest	ac.csg.pu.PromotionTest	9	0	0	0.024s	100%
TenthOrderLogicTest	ac.csg.pu.TenthOrderLogicTest	7	0	0	0.009s	100%

In addition to the two classes above, the following classes were also tested:

- Product – tested that VAT-exempt products keep their base price, that non-exempt products have 20% added correctly, and that all fields are stored and returned correctly
- OrderItem – tested that the total price is calculated as quantity multiplied by purchase price, that a single item's total matches its purchase price, and that a discounted item has a lower purchase price than its unit price
- Order – tested that a new order starts with no items, that items can be added and cleared, that the status can be updated, and that all fields match what was passed in on creation
- TenthOrderLogicTest – tested the logic that detects when a customer is placing their tenth order, checking counts of 0, 8, 9, 10, and 19, and also tested that the generated temporary password is non-null and exactly 10 characters long

4.2 Subsystem Testing

The purchasing subsystem was tested by running the application and placing orders through the GUI. The following tests were verified manually:

Test Case	Steps	Expected Output	Actual Result
Browse and add to cart	Log in as customer, open product list, add Ibuprofen to cart	Product appears in cart with correct price	Pass
VAT applied at checkout	Add a non-exempt product, view cart total	Total includes 20% VAT	Pass
Active promotion discounts item	Admin sets 10% off Omega 3, customer adds it to cart	Price shown as 10% less	Pass
Inactive promotion not applied	Admin disables promotion, customer adds same product	Full price shown, no discount	Pass
Complete a purchase	Customer fills in address and confirms order	Order appears in order history with status Accepted	Pass
Tenth order discount	Customer with 9 previous orders places another	10% discount applied automatically	Pass
Admin creates a promotion	Admin fills in promotion name, dates, and discount	Promotion appears in promotions list	Pass
Admin deactivates a promotion	Admin sets promotion to inactive	Promotion no longer applies at checkout	Pass
Invalid login rejected	Enter wrong password	Error message shown, no access granted	Pass
First login password change	New user logs in with generated password	Prompted to set a new password before continuing	Pass

All features behaved as expected during manual testing with no errors observed.

4.3 Required Interface Testing

This is a plan for how IPOS-PU would be tested against the other two systems.

For IPOS-SA, the main thing to test is whether stock levels update properly after an order is placed. When a customer completes a purchase, IPOS-PU sends a request to SA to reduce the stock for each item bought. To test this, we would check that the right request is being sent with the correct product IDs and quantities, and that if SA sends back an error the application handles it without crashing. A mock version of the SA server could be used so we can control what it returns and test different scenarios like a timeout or a rejected request.

For IPOS-CA, the main thing to test is whether products are being fetched and displayed correctly. IPOS-PU pulls its product data from CA, so we would check that when CA returns a list of products they show up properly in the application with the right names and prices. We would also test what happens if CA is unavailable, to make sure the app either shows a sensible error or falls back gracefully rather than just breaking.

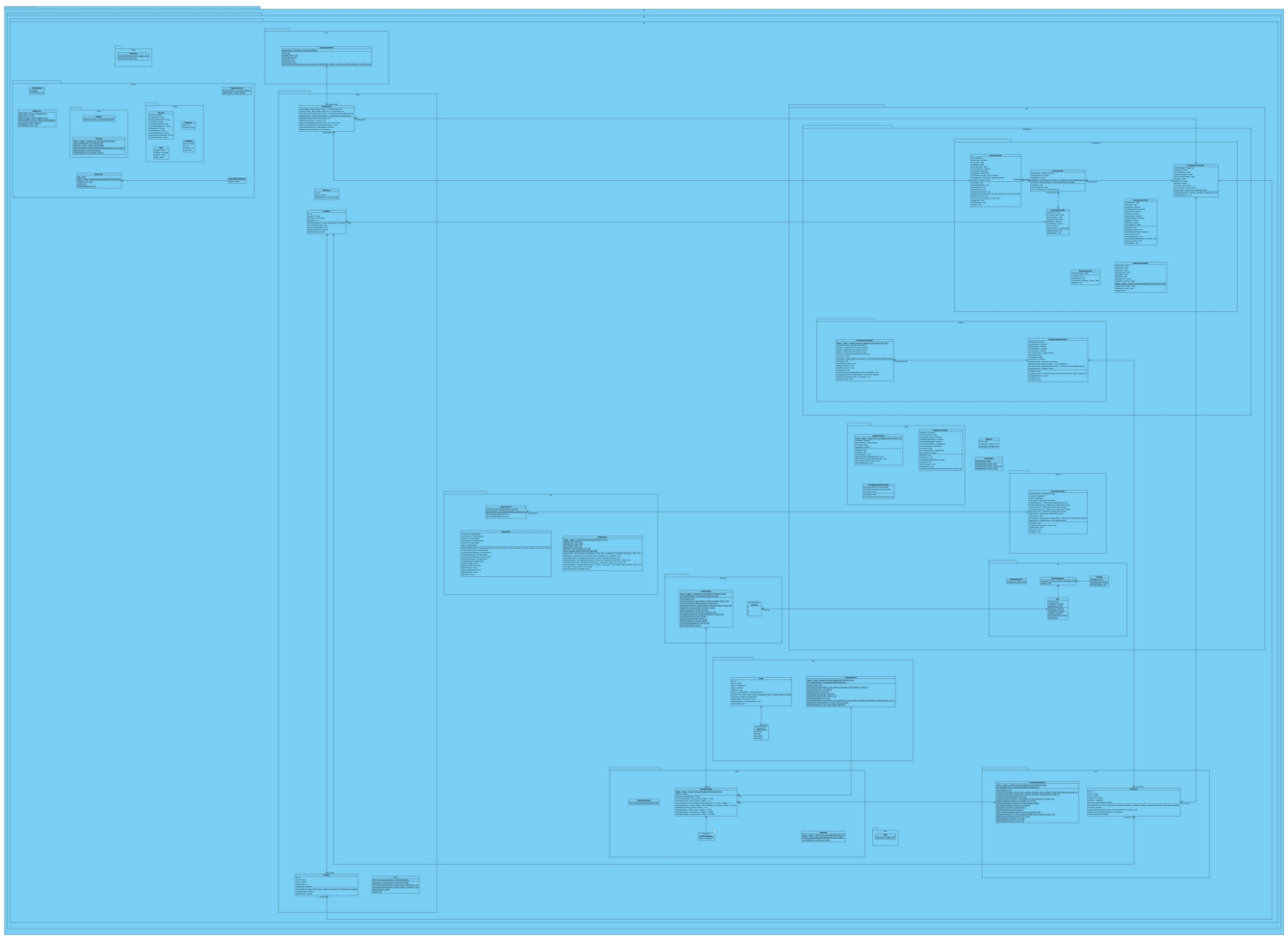
4.4 Non-Functional Testing

Security – we checked that passwords are not stored in plain text and that users cannot access screens they are not supposed to. For example, a customer logging in should not be able to reach the admin dashboard, and a commercial user should not see other merchants' products. New accounts are given a temporary password and the user is forced to change it on first login, which was verified manually.

Performance – the app was run with test data in the database and everything loaded quickly with no noticeable delays. Browsing products, viewing orders, and generating reports all responded near instantly under normal use. Since this is a single-user desktop application, performance under heavy load is not a major concern, but nothing slow was observed during testing.

Reliability – we went through all the main screens repeatedly to check for crashes or anything unexpected. The app behaved consistently throughout. Things like entering a wrong password or trying to check out with an empty cart were handled cleanly without any errors being thrown.

Appendices

Appendix A

A high resolution version of the below image is available in the project binder under: “Project Binder/04_Draft_Deliverables/04c_Week_13_Deliverables”